1일차 강의 자료 · 8시간

Cursor AI를 활용한 소프트웨어 개발

Prompt Engineering · Concept-to-Code Traceability · Dual-Track TDD · PRD

1교시	Prompt Engineering & Cursor AI 설치	P-C-T-F · 7단계 · ICL/CoT/ReAct
2·3교시	아름다운 SW — 해석학적 접근	C2C Traceability · 의사결정 파이프라인
4교시	Magic Square × C2C × 문제인식	온톨로지 · ECB 분석 · 문제인식 실습
5교시	Dual-Track TDD 방법론	왜 늦게 실패? · Problem · Scenarios
6교시	Magic Square — Dual-Track TDD 요구사항	UX Contract · Logic Rule 작성
7교시	Toy Project PRD + Cursor AI 개발 시작	PRD 작성 · AI Assist 코딩 시작
8교시	To-Do 구조 작성 원리	체크박스 · 속성 테스트 · 검증 포인트

2026 · 김대경 Ph.D.

1일차 시간표 (8시간)

일정

1교시	60분	Prompt Engineering & Cursor AI	P-C-T-F · 7단계 · Few-Shot · CoT · ReAct · Cursor 설치 및 Git
2교시	60분	아름다운 소프트웨어 — 해석학적 접근	C2C Traceability · 의사결정 파이프라인 · 문제인식 + 온톨로지 + 프롬프팅 체인
3교시	60분	(2교시 연속)	ECB 모델 · 4계층 구조 · Semantic Erosion 패턴
4교시	60분	Magic Square × C2C × 문제인식	Magic Square 소개 · ECB 분석 실습 · 문제인식 작성
// 점심 휴식 60분			
5교시	60분	Dual-Track UI+Logic TDD + Toy Project 소개	왜 늦게 실패? · Problem & Scenarios · Toy Project 소개
6교시	60분	Magic Square — Dual-Track 요구사항 작성	UX Contract · Logic Rule · 3단 매핑 표
7교시	60분	Toy Project PRD 작성 + Cursor AI로 개발 시작	Gherkin · PRD 구조 · Cursor Agent 활용
8교시	60분	To-Do 구조 작성 원리	체크박스·번호체계·Requirements 추적·속성 테스트·검증 포인트

01

Prompt Engineering & Cursor AI

고급 프롬프트 설계 기본기 · P-C-T-F · 7단계 · ICL/CoT/ReAct · Cursor 설치

1교시

60분

1.1 P-C-T-F 프레임워크 — 프롬프트 설계의 기본기

1교시

프롬프트 엔지니어링

프롬프트(Prompt)란?

인공지능에 지시하는 명령어 또는 AI와의 대화를 위한 질문, 지시, 요청.

더 넓게는 AI가 의도를 이해하고 적절한 응답을 생성하도록 안내하는 모든 형태의 입력.

마치 음악가가 악기를 다루듯,
작가가 펜을 다루듯,
프롬프트 엔지니어는 언어를 통해 AI의 잠재력을 이끌어냄.

좋은 프롬프트의 효과

왜 중요한가?

같은 AI라도 프롬프트를 어떻게 작성하느냐에 따라 전혀 다른 결과 제공.

나쁜 예:
'코드 짜줘'
→ AI: '어떤 코드를 원하시나요?'

좋은 예 (P-C-T-F):
'너는 Python 전문가 [Persona].
사용자 인증 시스템 개발 중 [Context].
JWT 기반 로그인 함수 작성 [Task].
타입 힌트 포함, 주석 한국어 [Format].'
→ 정확한 코드 즉시 생성

좋은 프롬프트의 6가지 핵심 원칙

Principles of Effective Prompting

1 명확성 (Clarity)

모호함 없이
정확하게 의도 전달

2 구체성 (Specificity)

필요한 세부 사항을
충분히 제공

3 맥락 제공 (Context)

배경 정보로
AI 상황 이해 지원

4 구조화 (Structure)

복잡한 요청은
논리적으로 구성

5 목적 정의 (Purpose)

무엇을, 왜
필요한지 명확히

6 제약 설정 (Constraints)

형식, 길이,
스타일 등 지정

프롬프트의 유형 분류

목적과 구조에 따른 분류

유형	설명	예시
질문형	정보 탐색, 설명 요청	머신러닝이란 무엇인가요?
지시형	특정 작업 수행 요청	Python으로 정렬 함수를 작성해줘
역할 부여형	페르소나 설정	너는 시니어 백엔드 개발자야
분석형	데이터·코드 분석	이 코드의 시간 복잡도를 분석해줘
창작형	새로운 콘텐츠 생성	소프트웨어 개발 원칙 5가지를 만들어줘
대화형	멀티턴 상호작용	이전 답변을 바탕으로 더 구체화해줘

1.1 P-C-T-F 프레임워크 — 프롬프트 설계의 기본기

1교시

"AI 응답 품질의 80%는 프롬프트 설계에 달려 있다 — 나쁜 예: '코드 짜줘' vs 좋은 예: P-C-T-F 완성 프롬프트"

P

Persona (페르소나)

AI에게 역할·전문성 부여
"너는 10년 경력의 Python 시니어 개발자야"

C

Context (맥락)

배경 상황 제공
"신규 스타트업의 사용자 인증 시스템을 개발 중이야"

T

Task (작업)

구체적 요청 명시
"JWT 기반 로그인 함수 3개를 작성해줘"

F

Format (형식)

출력 형식 지정
"타입힌트 포함, 한국어 주석, 에러처리 포함"

1.2 7단계 프롬프트 구성법 — 구조적 사고 형성

1교시

① **Role & Goal**

너는 기획 전문가 / 목표: 보고서 요약

② **Context & Input**

상황: 제조 문제인식 / 입력: 온톨로지 데이터

③ **Constraints**

300자 이내, 안전 규칙 준수

④ **Output Format**

표 형식으로 출력

⑤ **Example (Few-Shot)**

예시: 문제 → 해결 3가지

⑥ **Validation**

답변 전에 검토해줘

⑦ **Iteration**

부족하면 추가 질문해줘

프롬프트 기법 비교 정리

상황별 최적 기법 선택

기법	핵심 원리	사용 상황	효과
P-C-T-F	역할·맥락·작업·형식 4단계	일반 코딩, 분석 요청	★★★★☆
7-Step	목표~반복 7단계 통합	복잡한 기획, 설계 작업	★★★★★
KERNEL	간단·설명·정제·좁힘·탐색·학습	빠른 반복 개선	★★★★☆
Few-Shot	예시를 통한 패턴 학습	특정 형식 반복 출력	★★★★★
CoT	단계별 사고 유도	수학, 논리, 복잡 추론	★★★★★
ReAct	추론-행동-관찰 루프	에이전트 작업, 툴 사용	★★★★☆

KERNEL	영어	핵심 원칙
K	Keep simple	프롬프트를 **단순하게** 유지하라
E	Explain needs	필요한 것을 **명확히 설명**하라
R	Refine	결과를 보고 **반복적으로 개선**하라
N	Narrow	범위를 **좁혀서 초점**을 맞춰라
E	Explore	다양한 옵션을 **탐색**하라
L	Learn	예시를 통해 AI가 **패턴을 학습**하게 하라

1.3 고급 프롬프트 전략 — ICL · CoT · ReAct

1교시

ICL (In-Context Learning) 문맥 내 학습

예시를 프롬프트에 포함해 AI가
즉시 패턴 학습 — 모델 파인튜닝 없이
특정 형식·스타일 유도

Few-Shot의 상위 개념

Few-Shot 2~5개 예시 제공

입력→출력 예시 2~5개 제공
동일한 패턴으로 새 입력 처리
특정 코드 형식 반복 출력에 최적

효과 ★★★★★

CoT (Chain-of-Thought) 단계별 사고 유도

"단계별로 생각해줘" 추가
복잡한 계산·논리 문제에 필수
AI 추론 과정이 투명해짐

수학·논리 ★★★★★

ReAct (Reasoning + Acting) 추론-행동-관찰 루프

Thought→Action→Observation- → Thought→ 반복
에이전트 기반 작업·툴 사용에 특화

에이전트 작업 ★★★★☆

2026 조합 추천 P-C-T-F(기본) + Few-Shot(형식 통일) + CoT(복잡한 추론) + ReAct(에이전트 작업) → Cursor AI에서 그대로 활용 가능

Few-Shot Learning

예시를 통한 패턴 학습

Few-Shot 예시

원리:
예시를 제공하면 AI가 패턴을 학습하여
새로운 입력에 동일한 패턴 적용

예시 1 (덧셈):
입력: '두 수를 더하는 함수'
출력:

```
def add(a: int, b: int) -> int:
    return a + b
```

예시 2 (곱셈):
입력: '두 수를 곱하는 함수'
출력:

```
def multiply(a, b) -> int:
    return a * b
```

질문: '두 수를 나누는 함수'→ AI가 동일 패턴으로 답변

CoT & ReAct

Chain-of-Thought (CoT):
단계별 사고 유도

프롬프트:
'15% 할인 후 20% 추가 할인의
총 할인율을 단계별로 계산해줘'

AI 응답:
1단계: 10,000 × 0.85 = 8,500원
2단계: 8,500 × 0.80 = 6,800원
3단계: (10,000 - 6,800) / 10,000
답: 32% 할인

ReAct:
Thought → Action → Observation→ 반복 루프

사고 제어 기법 (Cognitive Control)

복잡한 문제 해결을 위한 고급 기법

기법	역할	프롬프트 예시
Chain-of-Thought	단계적 추론 유도	'단계별로 생각해서 답해줘'
Few-Shot	예시로 행동 패턴 유도	예시 2-3개 제공 후 질문
Extended Thinking	복잡한 문제에 충분한 시간 부여	'천천히 깊이 생각해줘'
Slow Thinking	가정 검토, 성급한 결론 방지	'결론 내리기 전에 가정을 검토해'
Self-consistency	여러 추론 결과 비교	'3가지 방법으로 풀고 비교해줘'
ReAct	행동-관찰-수정 루프	에이전트 기반 작업 처리

1일차 강의 자료 · 8시간

워크 북 실습

D1_1.실습- 실무 프롬프트 고치기 & 개선하기

프롬프트 기반 코딩의 진화

코딩 패러다임의 혁명적 변화

"AI가 코더를 대체하지는 않지만, AI를 활용하는 코더가 그렇지 않은 코더를 대체할 것입니다."

~2023

전통적 코딩

모든 코드를
수작업으로 작성

개발자가 직접
문법·API 암기

2024

AI 보조 코딩

GitHub Copilot,
Tabnine 등

AI가 부분적으로
코드 보완

2025~

AI 에이전트 코딩

Cursor, Cline

AI가 모든 코드를
완전 자동 생성

개발자 = 설계자

기존 개발 vs 바이브 코딩

기존 개발 방식

직접 코드 작성
한 줄씩 타이핑 → 문서 검색 → 스택오버플로우 반복

수동 디버깅
에러 메시지 → 검색 → 원인 추론 → 수정 → 반복

파일별 편집
에디터에서 파일을 열고 닫으며 컨텍스트 전환

긴 피드백 루프
코드 작성 → 빌드 → 테스트 → 수정 cycle이 느림

지식 검색
API 문서, 예제 코드를 직접 찾아보는 시간 낭비

바이브 코딩 방식

의도 중심 지시
"로그인 기능 만들어줘" → AI가 파일 생성·수정 자동화

AI 자율 디버깅
에러 발생 시 AI가 원인 분석 → 수정 → 검증 루프 자동

프로젝트 전체 이해
AI가 전체 코드베이스 파악 후 관련 파일 동시 수정

즉각적 피드백
지시 → 코드 생성 → 실행 → 결과 확인까지 수분 내 완료

검증 중심 역할
개발자는 '무엇을 만들지'에 집중, AI는 '어떻게'를 담당

주요 AI 코딩 에이전트 비교

2025년 기준

도구	특징	강점	가격
Cursor AI	VS Code 기반 IDE, 멀티파일 편집	Agent/Plan/Ask 3모드, .cursorrules	Free / $20~
GitHub Copilot	GitHub 공식 통합, 모든 IDE 지원	기업 보안 강함, 대형 레포 최적화	$10~$19/월
Cline	오픈소스, VS Code 확장	SNS 인기, 직접 API 사용 가능	Free (API 비용)
Amazon Q	AWS 생태계 최적화	AWS 서비스 연동, 기업용	$19/월
Claude (Anthropic)	Artifacts, Projects 기능	긴 컨텍스트, 문서 작업 특화	Free / $20~
Gemini CLI	Google	1M 토큰 컨텍스트, Google Drive 연동 , 무료 티어 넉넉	
OpenAI Codex CLI	OpenAI	GPT-4o 기반, 함수 단위 완성 강점, GitHub Copilot 연계	

Cursor AI 요금제 비교 (2025 기준)

가입 즉시 사용 가능, Pro 14일 무료 체험

요금제	가격	자동 완성	프리미엄 요청	비고
Free	무료	월 200~2000회	월 50회 (느림)	2주간 Pro 체험 포함
Pro	$20/월	무제한	월 500회 (빠름) + 느림 무제한	GPT-4, Claude 사용 가능
Business	$40/월/인	Pro 기능 전체	Pro + 조직용 보안·대시보드	팀 사용 추천

Cursor AI 주요 단축키

핵심 커맨드 모음

단축키	기능	설명
Tab	코드 자동 완성 수락	약 1초 가만히 있으면 Copilot++ 제안 표시
Cmd + K	인라인 프롬프트	현재 위치에서 AI에게 코드 생성·수정 요청
Cmd + L	AI Chat 패널 열기	우측 사이드바에서 대화형 AI 질문
Cmd + I	Composer (Agent)	전체 프로젝트 범위 멀티파일 작업
@파일명	파일 컨텍스트 참조	특정 파일을 프롬프트에 포함
@심볼	코드 심볼 검색	함수·클래스를 빠르게 찾아 참조

Python 실습 환경 구성

개발 환경 셋업 가이드

▶ **Python 설치 (권장: 3.10.11)**

- python.org에서 다운로드 → 설치 시 'Add to PATH' 필수 체크
- cmd에서 확인: python --version

▶ **가상 환경 생성 (venv 권장)**

- python -m venv py3_10_basic
- py3_10_basic₩Scripts₩activate (Windows)
- source py3_10_basic/bin/activate (Mac)

▶ **필수 라이브러리 설치**

- pip install pytest matplotlib pandas jupyter
- pip install pyqt5 (UI 실습용)

▶ **IDE 선택**

- Cursor AI (권장) | PyCharm Community (무료) | VS Code + Python 확장

소프트웨어 형상 관리(SCM)

Software Configuration Management

SCM 개요

SCM의 목적:

① 산출물 품질의 향상
② 개발 및 유지보수 생산성 향상
③ 사용자 요구사항 체계적 관리
④ 변경 추적 및 통보

SCM의 4대 활동:
① 형상 식별 (CI)
② 형상 통제 (CC)
③ 형상 이력관리 (CS)
④ 형상 평가 (CA)

버전 관리 시스템 종류

버전 관리 시스템 종류:

로컬 VCS:
단일 PC에서 관리
→ 협업 불가

중앙집중식 VCS:
SVN, CVS
→ 서버 장애 시 전체 중단

분산형 VCS (Git):
각 개발자가 전체 저장소 보유
→ 오프라인 작업 가능
→ 서버 없이 커밋
→ 브랜치 생성/전환 빠름

Git vs GitHub 비교

도구의 역할 이해

구분	Git	GitHub
성격	소프트웨어 (로컬 도구)	웹 서비스 (클라우드 플랫폼)
역할	버전 관리, 변경 추적	원격 저장소, 협업 허브
오프라인	가능 (모든 명령 로컬)	불가능 (인터넷 필요)
유사 서비스	없음 (표준 도구)	GitLab, Bitbucket, Gitea
창시자	Linus Torvalds (2005)	Tom Preston-Werner (2008)

핵심 Git 명령어

실무 필수 명령어 정리

명령어	기능	예시
git init	로컬 저장소 초기화	git init my-project
git clone	원격 저장소 복제	git clone https://github.com/user/repo.git
git add	변경 파일 스테이징	git add . | git add main.py
git commit	스냅샷 저장	git commit -m 'feat: 로그인 기능 추가'
git push	원격 저장소에 업로드	git push origin main
git pull	원격 변경사항 통합	git pull origin main
git branch	브랜치 생성/확인	git branch feature-login
git checkout	브랜치 전환	git checkout feature-login

Git 기본 워크플로우

Working Directory → Staging → Local → Remote

Working Directory	Staging Area	Local Repository	Remote Repository
파일 수정 작업 공간	git add 커밋 준비	git commit 스냅샷 저장	git push GitHub

git pull / git fetch (원격 → 로컬 동기화)

브랜치 관리 전략

Git Flow / Feature Branch

▶ 기본 브랜치 구조

- main: 배포 가능한 안정 버전
- develop: 개발 통합 브랜치
- feature/xxx: 기능별 개발 브랜치
- hotfix/xxx: 긴급 버그 수정

▶ 실무 워크플로우

- git branch feature-login # 브랜치 생성
- git checkout feature-login # 브랜치 전환
- ... 개발 ...
- git add . && git commit -m 'feat: 로그인 구현'
- git push origin feature-login
- # GitHub에서 Pull Request 생성 → 코드 리뷰 → Merge

▶ 커밋 메시지 컨벤션

- feat: 새 기능 | fix: 버그 수정 | docs: 문서
- refactor: 리팩토링 | test: 테스트 | chore: 기타

1.4 Cursor AI 설치 및 환경 점검 · Git 연동

1교시 실습

⚙️ Cursor AI 설치 체크리스트

- ☐ cursor.com 접속 → 다운로드
- ☐ VS Code 스타일 설정 가져오기
- ☐ 기본 모델: claude-3.7-sonnet 선택
- ☐ Cmd+K / Cmd+L / Cmd+I 동작 확인
- ☐ .cursorrules 파일 생성 연습
- ☐ Python 3.10+ & pip 확인
- ☐ git --version 확인
- ☐ GitHub 계정 로그인

🔑 Cursor 3가지 모드

Agent 🤖

복잡한 코딩·리팩토링
다중 파일 자율 수정

Plan 📋

복잡한 작업 계획 수립
코드 수정 없이 계획만

Ask 💬

핵심 단축키

Cmd+K 인라인 편집 · Cmd+L Chat 패널
Cmd+I Composer(Agent) · Tab 자동완성 수락

1일차 강의 자료 · 8시간

워크 북 실습

D1_2.실습- Git 기본 + Cursor 연동

02

아름다운 소프트웨어를 위한 해석학적 접근

Concept-to-Code Traceability · 의사결정 파이프라인 · 문제인식 + 온톨로지 + 프롬프팅 체인

2·3교시

120분

2.1 왜 설계 의도는 코드에서 사라지는가? — Semantic Erosion(의미침식)

2·3교시

"당신의 팀은 완벽한 요구사항 문서를 작성했다. 6개월 후 코드를 보니 설계 의도는 온데간데없고 스파게티만 남아 있다."

문서-코드 단절

개념 분석 산출물이
코드와 직접 연결 안 됨

테스트 수준 혼재

기술 단위 테스트만 작성
비즈니스 의미 검증 없음

UI의 도메인 잠식

UI 요구사항이 도메인
모델을 오염

아키텍처 우선 개발

아키텍처가 개념보다
먼저 결정됨

결과: "Semantic Erosion" — 시스템은 시간이 지날수록 의미적 일관성(Semantic Coherence)을 잃는다

도메인 로직 → UI 이동 · 컨트롤러 비대화 · 테스트가 구현 세부사항만 기술 · 원래 개념 모델 사라짐

2.2 문제인식(Problem Recognition) + 의사결정 파이프라인

2·3교시

문제인식이란: "언제 시스템이 실패했다고 말해야 하는가?"를 정의 — "무엇을 만들까"가 아닌 실패 조건부터

의사결정 파이프라인 전체 흐름

문제인식 → 의미화 → 온톨로지 → 디지털화 → 논리화 → 정규화 → 검증화 → 추상화 → 최적화 → 실행·결정

❌ 해결책 중심 (나쁜 문제인식)

- 책을 빌리는 기능이 필요하다
- 로그인 화면을 만들자
- 검색 기능을 추가하자

✅ 실패 조건 중심 (올바른 문제인식)

- 연체·한도 초과 시 → 대출 거부
- 이메일 오류·비밀번호 오류 각각 처리
- 검색어 0자·특수문자·결과 없음 정의

2.3 온톨로지(Ontology) — ECB 분석 모델로 세계를 구조화

2·3교시

온톨로지: "우리가 세계를 어떤 개념과 관계로 나누어 이해할 것인가"를 정의하는 명시적 모델 — 팔란티어의 성공 비결
예: '직원(객체) ' 이 ' OO 코리아(조직)'에서 근무한다
주요 온톨로지 예시

- 지식 그래프 및 의미망: '김철수(35세, 남, 테슬라 코리아 소속)'와 같이 객체 간의 복합적 관계를 그래프 형태로 표현하여 AI가 맥락을 이해하게 함.
- 분야별 온톨로지: 의학(질병, 증상, 약물 간 관계), 전자상거래(상품, 카테고리, 리뷰 데이터 간 관계), 에너지 산업(유정, 엔지니어, 상태 정보 간 공통 관점).
- 구조적 정의: "A는 B의 일종이다(is-a)", "A는 B를 가지고 있다(has-a)"와 같은 명세서.

<<Entity>>

Entity (엔티티)

시스템이 장기적으로 관리하는 도메인 정보

예: Member · Book · Loan · Order

💡 판단하지 않는다. 상태를 유지한다.

<<Control>>

Control (제어)

유스케이스를 수행하는 흐름 제어·업무 조정

예: LoanPolicy · LoginControl · OrderService

💡 판단한다. 조율한다. 상태는 최소화.

<<Boundary>>

Boundary (경계)

시스템과 외부 액터 사이의 접점

예: LoanUI · LoginAPI · ExternalPayment

💡 비즈니스 규칙을 포함하지 않는다.

⚠️ ECB는 MVC·Spring Controller와 다르다. 분석 개념 도구이지, 코딩 규칙이 아니다 (Ivar Jacobson, OOSE 1992)

2.4 Concept-to-Code Traceability — 정의와 4계층 모델

2·3교시

C2C Traceability 정의

문제 영역에서 식별된 개념(Concept)과 코드(Code) 사이의 의미적·구조적·행위적 연결이 명시적으로 유지·검증·진화 가능한 상태 — 의미 보존(Semantic Preservation) 목표

Layer 1	CONCEPT LAYER	*"무엇이 존재하는가"* ECB 개념 모델 · 역할 분류	↕ 의미 정의
Layer 2	RESPONSIBILITY LAYER	*"누가 언제 무엇을"* 유스케이스 · 시나리오 · 협력	↕ 행위 구조
Layer 3	TEST LAYER	*"개념이 보존되었는가"* Acceptance Test + Logic Test (Dual-Track)	↕ 의미 검증
Layer 4	CODE LAYER	*"어떻게 구현되었는가"* ECB 스테레오타입 반영 구현	↕ 개념 구체화

핵심 성질: 완전성(Completeness) · 일관성(Consistency) · 검증 가능성(Verifiability) · 진화 가능성(Evolvability)

2.5 프롬프팅 체인 — C2C Traceability의 현대적 구현

2·3교시

핵심 통찰: 프롬프트 엔지니어링 = C2C Traceability를 구현하는 현대적 방법론

프롬프트 = 명시적 의도 기록(Traceability Documentation) + 맥락 보존(Context Preservation) + 재생성 가능성(Reproducibility)

Layer 1

ECB 개념 분석

ECB 분석: 도서관 시스템
ENTITIES: Member, Book, Loan
CONTROLS: LoanPolicy, LoanService
BOUNDARIES: LoanUI, LoanAPI
각 개념의 ECB 스테레오타입과
책임을 정의하라

→

Layer 2

책임 시나리오

LoanPolicy(Control)의 책임:
1. 대출 가능 여부 판단
2. 대출 기간 계산
Trace: LoanPolicy는 Concept Layer의 Control

→

Layer 3

테스트 명세

LoanPolicy.can_borrow() 테스트:
- 정회원 + 0권 → True
- 정회원 + 3권 → False
Trace: Responsibility Layer의
"대출 가능 여부 판단"

→

Layer 4

코드 구현

위 테스트를 통과하는 LoanPolicy 구현
ECB: <<control>>
Trace: Test→Responsibility →Concept

각 프롬프트가 상위 계층을 명시적으로 참조 → Chain-of-Prompts = Traceability Chain → Traceability 자동 확보

2.6 ECB(Entity-Control-Boundary) 분석 패턴

객체 역할 기반 설계

E Entity 엔티티

시스템이 장기적으로 보관해야 하는 데이터·정보

특징:
- 데이터 보유 (속성)
- 비즈니스 규칙 포함
- DB 테이블과 매핑

예시:
User, Product, Order
BookInfo, Inventory

B Boundary 경계

시스템과 외부 액터(사람·시스템) 사이의 인터페이스

특징:
- UI 화면, API 엔드포인트
- 입출력 검증
- 외부와의 통신

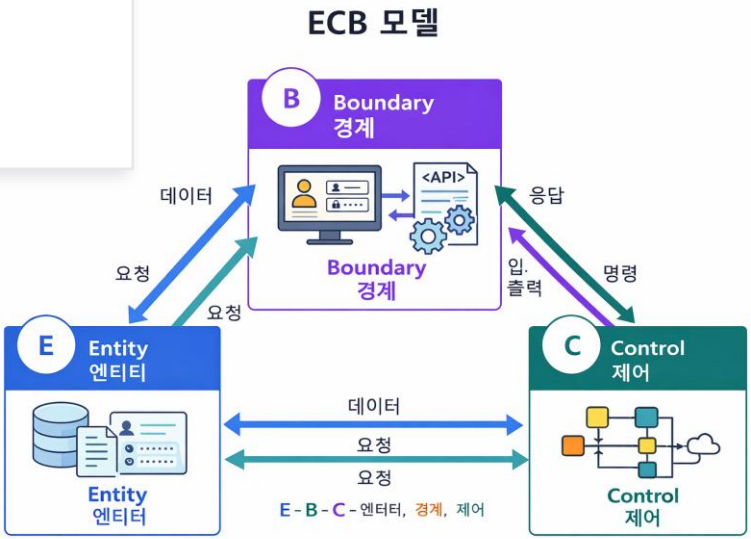
예시:
LoginForm, APIController
DashboardView, WebSocket

C Control 제어

유스케이스 흐름을 조정하는 비즈니스 로직

특징:
- Entity와 Boundary 연결
- 비즈니스 규칙 실행
- 트랜잭션 관리

예시:
AuthService, OrderProcessor
BookSearchController



2.7 역할 기반 접근 제어(RBAC) — 개념과 ECB 연결

2·3교시

ECB에서 Control이 비즈니스 규칙을 실행하고 Boundary가 외부와 통신한다면 — RBAC는 "누가 어떤 Control을 실행할 수 있는가"를 결정하는 접근 제어 레이어입니다

🔗 ECB 패턴에 RBAC를 더하면

E — Entity

User·Role·Permission·Resource
→ RBAC 핵심 데이터 보관

B — Boundary

로그인 폼·API 엔드포인트
→ Role 정보를 받아 Control로 전달

C — Control

AuthService·PermissionChecker
→ Role-Permission 매핑·접근 허용/거부

📐 RBAC 4대 구성 요소

User	시스템 사용자 — 개발자·운영자·관리자
Role	권한 묶음 단위 — admin·developer·viewer
Permission	자원에 대한 행동 — read·write·delete
Resource	접근 대상 — DB·API·파일·서비스

📌 용어 구분: 큰 틀 vs 세부 표현

큰 틀	→ *역할 기반 접근 제어 (RBAC)*
메뉴·UI	→ *권한(역할)에 따른 메뉴 가시성 제어*
내비게이션	→ *RBAC에 따른 내비게이션 제어*

2.8 RBAC 설계 패턴 — 권한에 따른 메뉴 가시성 제어

2·3교시

"권한에 따른 메뉴 가시성 제어" = Boundary(UI)가 Control(AuthService)에 Role을 질의해 메뉴를 보여줄지 숨길지 결정 — UX Contract의 '보인다/안 보인다' 언어와 직결

📊 Role-Permission 매트릭스 (내비게이션 메뉴 기준)

역할	대시보드	사용자관리	설정	삭제
admin	✅	✅	✅	✅
developer	✅	✅	❌	❌
viewer	✅	❌	❌	❌
guest	⚠️	❌	❌	❌

✅ UX Contract 연결 — "보인다 / 안 보인다" 규칙

Role=admin:	사용자관리 메뉴 보인다 · 삭제 버튼 활성화
Role=developer:	사용자관리 메뉴 보인다 · 삭제 버튼 비활성화
Role=viewer:	대시보드만 보인다 · 모든 관리 메뉴 숨김

Python — 메뉴 가시성 제어 구현

```python
from enum import Enum

class Role(Enum):
    ADMIN     = 'admin'
    DEVELOPER = 'developer'
    VIEWER    = 'viewer'

# 역할별 메뉴 가시성 정의
MENU_VISIBILITY = {
    Role.ADMIN: {
        'dashboard': True,
        'user_mgmt': True,
        'settings':  True,
        'delete_btn':True,
    },
    Role.DEVELOPER: {
        'dashboard': True,
        'user_mgmt': True,
        'settings':  False,
        'delete_btn':False,
    },
    Role.VIEWER: {
        'dashboard': True,
```

2.8 RBAC 설계 패턴 — RBAC + TBAC 하이브리드 모델

2·3교시

R

RBAC (Role-Based Access Control) — 역할 기반 접근 제어

Superadmin, Admin, Manager, User, Viewer 등 역할에 따라 기능 접근을 제어

예: Admin만 사용자 관리 가능, Viewer는 읽기만 가능

권한(permission) 기반 작업 시 유지보수 수월

T

TBAC (Team-Based Access Control) — 팀 기반 접근 제어

사용자가 소속된 팀에 따라 데이터 접근 범위를 제어

예: R&D 팀 소속 사용자는 R&D 팀 게시판/문서만 접근 가능

A

ABAC (Attribute-Based Access Control, 속성 기반 접근 제어)라고도 합니다. 팀 소속이라는 "속성"을 기준으로 접근 제어

2.9 RBAC × TDD × Cursor AI — 실전 워크플로우

2·3교시

RBAC를 TDD로 구현할 때 Cursor AI를 활용하는 전체 흐름 — C2C Traceability: 역할 개념 → 실패 테스트 → 최소 구현 → 리팩토링까지 의미 소실 없이 추적

① 역할 정의 (RED 준비)

ECB Entity로 Role·Permission 데이터 구조 설계

"RBAC를 위한 User·Role·Permission Entity를 설계해줘"

→

② 실패 테스트 작성 (RED)

권한 거부/허용 실패 테스트 먼저 → pytest FAIL 확인

"viewer가 delete 하면 403 에러 테스트 먼저 써줘"

→

③ 최소 구현 (GREEN)

메뉴 가시성 딕셔너리 + is_visible() 함수

"테스트 통과하는 최소 코드만 구현해줘"

→

④ 리팩토링 (REFACTOR)

DB 기반 동적 권한으로 개선 역할 계층 구조

"JWT claim에서 Role 읽어오는 구조로 리팩토링"

→

⑤ Git 커밋 & 추적

C2C Traceability Role개념→테스트 →코드 연결 확인

```
git commit -m
"feat: RBAC
메뉴 가시성 제어"
```

🔴 RED 단계 — 실패 테스트 예시

```
def test_viewer_cannot_see_user_mgmt():     # 메뉴 가시성
    assert not is_visible(Role.VIEWER, 'user_mgmt')

def test_admin_can_see_all_menus():         # admin 전체 접근
    assert all(is_visible(Role.ADMIN, m) for m in
MENU_VISIBILITY[Role.ADMIN])
```

📌 용어 정리

- 역할 기반 접근 제어(RBAC)
- 권한(역할)에 따른 메뉴 가시성 제어
- RBAC에 따른 내비게이션 제어

04

Magic Square × C2C × 문제인식

D1_4.실습- 간단한 앱 설계 (3계층)

마방진으로 배우는 Concept-to-Code Traceability 실전 적용

4교시

60분

4.1 Magic Square (마방진) 프로젝트 소개

4교시

Magic Square = 모든 행·열·대각선의 합이 같은 4×4 숫자 배열 — 두 빈칸 위치를 찾고 1~16 숫자 배치

4×4 Magic Square

16	3	2	13
5	10	11	?
9	6	?	12
4	15	14	1

합 = 34

ECB 개념 분석 (문제인식 → 설계)

<<Entity>>

MagicSquare — 4×4 격자 데이터 · Cell — 각 칸 값·위치·상태 · SolveResult — 해결과

<<Control>>

SquareValidator — 합 유효성 검증 · MissingFinder — 빈칸 위치 탐색 · Solver — 완성 알고리즘

<<Boundary>>

GridUI — 격자 시각화 · InputHandler — 사용자 입력 · ResultDisplay — 결과 표시

실습 목표: ECB 분류 완성+ 문제인식(실패 조건) 3가지 이상 도출

4.2 Magic Square — 문제인식 & C2C 연결 실습

4교시 실습

"해결책이 아닌 실패 조건을 먼저 정의하라" — 아래 표를 완성하는 것이 4교시 핵심 실습입니다

개념 (ECB)	책임 (Responsibility)	실패 조건 (Problem Recognition)	테스트 힌트
SquareValidator <<control>>	행·열·대각선 합 검증	합이 34가 아닐 때 → 유효하지 않은 배열	`is_valid() == False`
MissingFinder <<control>>	빈칸 위치 탐색	빈칸이 0개 또는 3개 이상 → 탐색 불가	`find_missing() raise`
MagicSquare <<entity>>	격자 데이터 보관	1~16 범위 초과 값 → 데이터 무결성 위반	`valid_range() == False`
GridUI <<boundary>>	격자 시각화 및 입력	입력값이 정수가 아닐 때 → UI 거부	`validate_input()`

실습 과제: 위 표를 완성하고 "빈칸이 대각선에만 있을 때" 추가 실패 조건을 스스로 도출해보세요

1일차 강의 자료 · 8시간

워크 북 실습

D1_3.실습- The Mom Test + 문제 10개 찾기
D1_4.실습- 간단한 앱 설계 (3계층)

05

Dual-Track UI + Logic TDD 개발 방법론과 요구사항

왜 우리는 항상 늦게 실패하는가? · Problem & Scenarios · Toy Project

5교시

60분

5.1 왜 우리는 항상 늦게 실패하는가? — 요구사항의 구조적 함정

5교시

"실패 조건이 정의되지 않으면, 성공은 합의될 수 없다"

기획서에는 기능이 있지만 실패 조건이 없다. 테스트는 항상 구현 후에 추가된다. 이것이 "늦은 실패"의 본질이다.

원인 1

기획은 있는데 설계는 없다

요구사항 문서 = 기능 목록
"실패 조건"이 없어 개발자마다 해석이 다름
→ 구현 후 "이게 아닌데" 반복

원인 2

테스트는 항상 늦게 추가된다

"구현 먼저, 테스트는 나중에"
→ 테스트가 구현의 하인이 됨
→ 비즈니스 의미를 잃음

원인 3

UI/Logic 분리의 오해

"UI 따로, 로직 따로" 개발
→ UI가 비즈니스 규칙 포함
→ 어느 것도 독립 테스트 불가

전통적 방식: 기획서→개발자 해석→구현→문제 발견 VS Dual-Track TDD: 실패 조건 합의→RED 테스트→구현

5.2 PRD Is the Code — Problem Recognition & Scenario (L0~L3)

5교시

"PRD는 문서가 아니라 테스트의 원천(Source of Truth)이다" — Problem + Scenario(L0~L3) + UX Contract + Logic Rule

PRD의 핵심 = 기능 목록이 아니라 실패 조건 목록

❌ 해결책 중심 PRD

- 로그인 기능을 만들자
- 책 검색 기능 추가
- 대출 한도 관리

✅ 실패 조건 중심 PRD

- 이메일 형식 오류 → 즉시 에러 표시
- 대출 3권 초과 → 거부 + 이유
- 연체 중인 회원 → 모든 대출 차단

② Scenario Level 0~3 — 실패를 시간축에 배열하다

L0

기능 개요

"회원이 책을 빌린다"

L1

Happy Path

정회원+ 가용 책→ 대출 성공

L2

Edge Case

대출 2 권→ 성공 / 3 권→ 거부

L3

Failure Path

연체 중·시스템 오류·권한 없음

5.3 Toy Project 소개 — Magic Square 완전 구현

5교시

Toy Project: Magic Square (4×4 마방진) — 오늘 8교시를 통해 문제인식→PRD→코드까지 완성하는 학습 프로젝트

4교시
문제인식 & ECB 분석
실패 조건 도출
ECB 분류

→

5교시
Problem & Scenarios
L0~L3 시나리오
UX/Logic 분리

→

6교시
Dual-Track 요구사항
UX Contract
Logic Rule 표

→

7교시
PRD 작성 + 개발 시작
Gherkin 기반
Cursor AI 활용

→

8교시
To-Do 구조 작성
체크박스·속성
검증 포인트

🎯 최종 목표: Claude Desktop 또는 PyQt5 UI + pytest 통과 + GitHub Push — C2C Traceability 완전 구현

06

Magic Square — Dual-Track UI + Logic TDD 기반 요구사항 작성

UX Contract · Logic Rule · 3단 매핑 표

6교시

60분

6.1 Dual-Track UI + Logic TDD — 핵심 원리

6교시

"UI는 디자인으로 RED를 만들고, 로직은 규칙으로 RED를 만든다 — GREEN과 REFACTOR는 항상 함께 간다"

UI Track (Boundary 검증)

```
test("입력 전 에러 없음", () => {
  render(<MagicSquareGrid />);
  expect(
    queryByText("오류")
  ).not.toBeInTheDocument();
});
// ✅ UX Contract 기반 RED
```

Logic Track (Control+Entity 검증)

```
def test_invalid_magic_square():
  ms = MagicSquare([[1,2,3,4],
    [5,6,7,8],[9,10,11,12],
    [13,14,15,16]])
  assert not ms.is_valid()
  # ✅ Logic Rule 기반 RED
```

⚠️ 두 테스트는 완전히 독립적 — UI 테스트는 로직을 모르고, Logic 테스트는 UI를 모른다

6.2 UX Contract · Logic Rule · 3단 매핑 표 작성

6교시 실습

① UX Contract 언어

- ✅ 보인다 / 안 보인다
- ✅ 가능하다 / 불가능하다
- ✅ 활성화 / 비활성화
- ✅ 포함한다 / 포함하지 않는다

② Logic Rule 언어

- ✅ 허용한다 / 거부한다
- ✅ 유지한다 / 중단한다
- ✅ 반환한다 / 차단한다
- ✅ 계산한다 / 저장한다

③ Magic Square 3단 매핑 표

Scenario	UX Contract	Logic Rule
모든 행 합이 34가 아닌 경우 (L3)	"오류 표시"가 보인다	합 검증이 거부된다
빈칸이 2개가 아닌 경우 (L3)	"빈칸 오류"가 보인다	탐색이 중단된다
올바른 숫자 입력 + 완성 (L1)	"완성!" 메시지가 보인다	해가 반환된다
입력값이 정수 아닌 경우 (L2)	입력 필드가 빨간색으로 표시	입력이 차단된다

⚠️ 이 동사들로 끝나지 않는 To-Do는 테스트로 변환하지 말 것 — 판단(Decision)을 포함한 것만 변환 대상

6.3 비교 — 전통적 개발 vs Concept-to-Code Traceability + Dual-Track

비교 항목	전통적 개발	일반 TDD	C2C + Dual-Track TDD
개념 명시성	낮음	낮음	높음 (ECB 스테레오타입)
책임 추적성	없음	약함	강함 (4계층 연결)
실패 조건 정의	늦음 (구현 후)	보통	먼저 (PRD → RED)
UI/Logic 분리	혼재	단일 루프	완전 분리 + Lock-Step
변경 영향 분석	어려움	보통	용이 (Trace 역추적)
도메인 의미 보존	불안정	불안정	안정적 (Drift 감지)
AI 협업 친화성	낮음	보통	높음 (Prompt Chain)
학습 곡선	낮음	보통	처음엔 높음 → 이후 낮음

📌 **선택 기준: 개념과 의도를 6개월 후에도 코드에서 읽을 수 있어야 한다면 → C2C Traceability + Dual-Track TDD**

"아름다운 소프트웨어란 기능이 동작하는 것이 아니라, 설계 의도가 코드 속에서 살아있는 것이다"

07

Toy Project PRD 작성 + Cursor AI로 개발 시작

AI Assist 기반 소형 프로젝트 시작 · Magic Square PRD 완성

7교시

60분

7.1 PRD 표준 구조 — Magic Square 작성 가이드

7교시

1 Overview (개요)

- 프로젝트 목적 (마방진 완성 지원)
- 문제 정의 (두 빈칸 찾기 + 배치)
- 성공 지표 (테스트 통과율, 성능)

2 Functional Requirements

- Gherkin 기반 Scenario (L0~L3)
- UX Contract (보인다/안 보인다)
- Logic Rule (허용/거부)

3 Non-Functional Requirements

- 성능: 빈칸 탐색 < 100ms
- 보안: 입력 유효성 검증 필수
- 확장성: 5×5 마방진 고려

4 Technical Constraints

- Python 3.10+, pytest, PyQt5
- ECB 3계층 아키텍처
- Git Flow + Cursor AI

5 Out of Scope

- 네트워크 기능 제외
- 다중 사용자 동시 접속 제외
- 5×5 이상 마방진 (Phase 2)

💡 좋은 PRD의 5조건: 명확성 · 측정 가능성 · 달성 가능성 · 관련성 · 완결성 (예외 케이스 포함)

7.2 Gherkin 시나리오 작성 — Magic Square 예시

7교시

Gherkin = Given-When-Then 형식의 시나리오 — 기획자·개발자·QA·Cursor AI 모두가 공유하는 공통 언어

Scenario 1: 정상 마방진 (L1 Happy Path)

```
Feature: 마방진 검증

Scenario: 올바른 4×4 마방진 검증
  Given 모든 행·열·대각선 합이 34인 배열
  When  is_valid() 호출
  Then  True 반환
```

Scenario 2: 실패 케이스 (L3 Failure Path)

```
Scenario: 빈칸이 없는 경우 탐색 실패
  Given 빈칸이 0개인 완성된 배열
  When  find_missing() 호출
  Then  MissingError 발생

Scenario: 범위 외 숫자 입력
  Given 17 이상 숫자 포함 배열
  When  validate_range() 호출
  Then  ValidationError 발생
```

7.3 Cursor AI로 개발 시작 — AI Assist 기반 워크플로우

7교시 실습

1 PRD → .cursorrules

.cursorrules에 ECB 구조·테스트 규칙 명시
→ Cursor가 항상 이 규칙 기반 코드 생성

→

2 Gherkin → RED Test

시나리오를 Cursor Ask에 붙여넣기
→ "이 시나리오 기반 pytest RED 테스트 생성"

→

3 RED → GREEN

Cursor Agent: "테스트를 통과하는 최소 구현 코드를 작성해줘"

→

4 GREEN → REFACTOR

Cursor Plan: "코드 품질 개선 계획
→ Agent로 실행"

→

5 Git Push

git add . && commit → push
Cursor Chat: "커밋 메시지 추천해줘"

.cursorrules 핵심 설정

project: MagicSquare · style: PEP8 + 타입힌트 · testing: pytest AAA패턴 · architecture: ECB(boundary/control/entity) · forbidden: print()디버깅

7.4 Cursor Rules — AI를 Dual-Track TDD 집행자로 만들다

Cursor Rules 철학: "프롬프트 한두 줄이 아니라, AI를 TDD 실행 엔진으로 고정하기 위한 강제 규칙 세트"

Cursor Rules = Concept-to-Code Traceability를 AI 협업에서 자동화하는 메커니즘

RED 선언 원칙

코드 작성 전 반드시 실패 테스트 먼저

UX Contract 또는 Logic Rule 기반으로만

ECB 무결성

Entity: 비즈니스 데이터만

Control: 조율과 규칙만

Boundary: 인터페이스만

Traceability 주석

모든 클래스에 ECB 스테레오타입 명시

Concept Trace + Responsibility 포함

Lock-Step REFACTOR

모든 테스트 GREEN 확인 후 리팩토링

UX 행동 변화 금지

실전 Cursor 프롬프트 패턴

RED 시작

```
We start with UI/UX RED.
Write failing UI tests based on
Figma behavior.
Do NOT implement yet.
```

GREEN 연결

```
Logic GREEN first.
Implement domain logic only to
satisfy logic tests.
No UI changes.
```

REFACTOR

```
All tests pass.
Refactor UI and logic together.
Do not change UX behavior.
Maintain ECB boundaries.
```

7.5 Cursor AI로 개발 시작 — AI Assist 기반 워크플로우

7교시 실습

.mdc 例

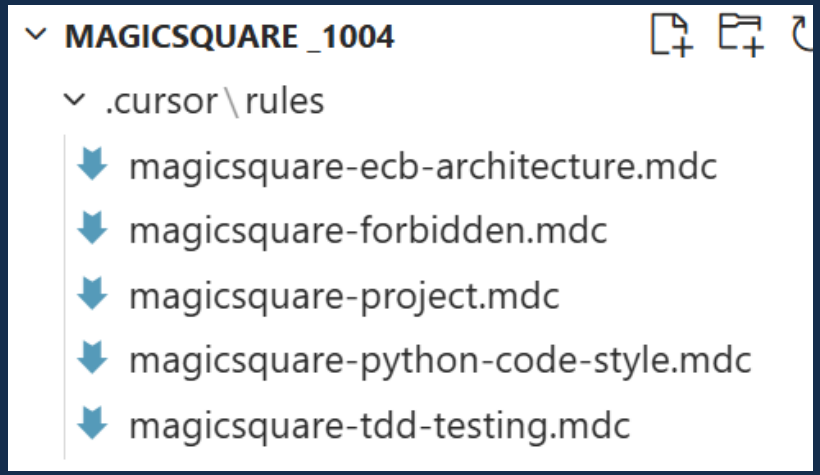
1. 프로젝트 전용 룰 (Project Rules) - 권장
특정 프로젝트에만 적용되는 룰입니다. 프로젝트의 최상위 루트 디렉토리에 파일로 생성합니다.
파일 위치: 프로젝트 폴더 가장 최상위(.cursor/rules)
파일명: .cursorrules (또는 .cursor/rules/*.mdc)
설정 방법: 프로젝트 폴더 내에 .cursorrules 파일을 만들고, AI가 지켜야 할 코드 스타일, 네이밍 규칙, 기술 스택 등을 텍스트로 작성하면 됩니다.
2. 글로벌 룰 (User Rules) - 공통
모든 프로젝트에 공통으로 적용되는 룰입니다.
위치: 커서 설정 화면 (Cursor Settings > General > Rules for AI)
설정 방법:
커서 실행
우측 상단 설정 버튼 클릭
General 탭 내 Rules for AI 입력창에 규칙 작성

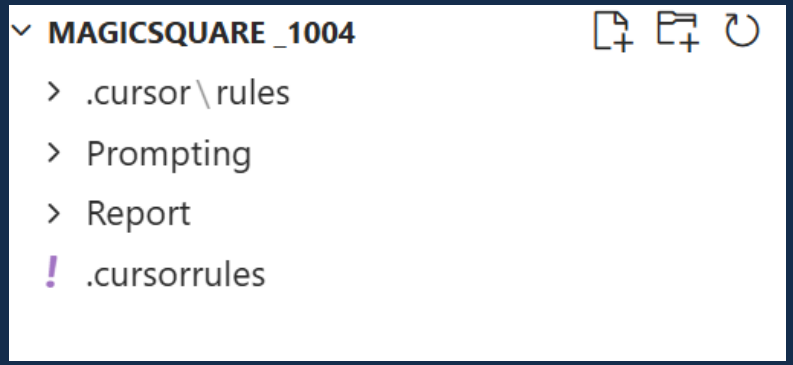
7.5 Cursor AI로 개발 시작 — AI Assist 기반 워크플로우

7교시 실습

.mdc 例

프로젝트\.cursor\rules\.mdc





1일차 강의 자료 · 8시간

워크 북 실습

D1_5. .cursorrules 작성

08

To-Do 구조 작성 원리

체크박스 · 단계별 번호 · Requirements 추적 · 체크포인트 · 속성 테스트 · 검증 포인트

8교시

60분

8.1 To-Do List vs Work Order — 역할 구분

8교시

핵심 전제: "모든 to-do는 테스트로 변환되지 않는다. 판단(Decision)을 포함한 to-do만 테스트로 변환된다"

구분	To-Do List	Work Order
목적	해야 할 일 목록화	LLM에게 실행 지시
수준	개략적, 의도 중심	구체적, 계약 중심
예시 내용	"마방진 검증 구현"	def is_valid(grid:List[List[int]])->bool: 반환, pytest AAA패턴
사용 시점	기획·분석 단계	개발·구현 단계
효과	우선순위 정리	AI 할루시네이션 방지

Work Order 예시 (Magic Square)

WI-001: SquareValidator.is_valid(grid) 구현 · ECB: <<control>> · 입력: 4×4 List[List[int]] · 반환: bool
실패조건: 합≠34 → False · 테스트: pytest AAA · 검증: test_invalid_magic_square() PASS

8.2 Cursor AI To-Do 리스트 생성 요청 프롬프트 템플릿

8교시

① 나쁜 프롬프트 vs ② 좋은 프롬프트 — C2C Traceability 기반 Work Order 생성

❌ 나쁜 프롬프트

```
"Magic Square TDD로 만들어줘"

문제점:
- 테스트 통과 기준 없음
- ECB 구조 미지정
- 어떤 to-do가 테스트인지 불명
- 검증 방법 없음
- Hallucination 위험 높음
- 범위 불명확 → 과도한 구현
```

✅ 좋은 프롬프트 (Work Order 구조)

```
## Context
TDD RED→GREEN 전환 단계
ECB: MagicSquare<<entity>>

## Test (첨부: @test_magic.py)
def test_invalid_square(): ...

## 제약 조건
1. 테스트 통과 최소 코드만
2. 타입힌트 필수
3. 하드코딩 허용 (GREEN)
4. SOLID 불필요 (REFACTOR)

## 요청: magic_square.py 작성
```

8.3 사용자 여정(User Journey) → To-Do 구조화

8교시

💡 핵심 전제: "To-Do는 사용자가 시스템과 상호작용하는 순간에서 나온다 — 행위가 없으면 To-Do도 없다"

🗺 사용자 여정이란?

1 진입점 (Entry)

사용자가 시스템에 접근하는 첫 행위
예: 4×4 Magic Square 게임 시작

↓

2 핵심 행위 (Core Action)

목적을 달성하기 위한 주요 상호작용
예: 숫자 입력 → 유효성 검사 요청

↓

3 결과 확인 (Outcome)

시스템이 반환하는 피드백 수신
예: 합계 34 성공 / 오류 메시지

📝 여정 → To-Do 변환 규칙

사용자 여정 문장	→ To-Do 항목
사용자가 ___ 한다	→ [ ] ___을 구현한다
시스템이 ___ 을 보여준다	→ [ ] ___ UI 컴포넌트 생성
___ 이 실패하면	→ [ ] 실패 케이스 테스트 작성
___ 을 확인한다	→ [ ] 검증 로직 + AAA 테스트

◆ MagicSquare 여정 → To-Do 예시

```
사용자가 4×4 격자에 숫자를 입력한다
→ [ ] GridUI <<boundary>> 4×4 테이블 구현
→ [ ] SquareValidator.is_valid(grid) 구현
```

✅ 여정 문장에 동사가 있어야 To-Do가 생긴다 — 동사 없는 여정 → 기획 메모 (To-Do 아님) To-Do: Work Order: 1:N

8.4 PRD 작성 → Work Order 자동 생성 구조

8교시

💡 **핵심 전제: "PRD는 판단(Decision)을 포함한 To-Do만 Work Order로 변환하는 필터다"**

📄 PRD 5요소 → To-Do 연결 구조 (MagicSquare 기준)

① 목표 (Goal)

4×4 Magic Square 유효성 검사 및 빈칸 탐색 구현

→ To-Do

[ ] SquareValidator 모듈 구현

② 사용자 (Who)

퍼즐 게임 플레이어 — 4×4 격자 숫자 입력

→ To-Do

[ ] GridUI <<boundary>> 설계

③ 기능 (What)

합계 검증 / 빈칸 탐색 / 결과 표시

→ To-Do

[ ] is_valid() find_missing() display()

④ 제약 (Constraint)

ECB 레이어 준수
pytest 커버리지 80%+

→ To-Do

[ ] .cursorrules 적용 + 테스트 선행 작성

⑤ 검증 (Acceptance)

합≠34 → False
빈칸=0 → raise ValueError

→ To-Do

[ ] AAA 패턴 테스트 케이스 명세

⚙️ To-Do → Work Order 변환 필터 (판단 포함 여부 기준)

PRD To-Do 목록 → 판단(Decision) 포함 여부 체크 → Work Order 생성

NO → 단순 메모 (보류)

✅ ④제약 + ⑤검증에 판단이 있어야 Work Order가 생성된다

Work Order 예시 (MagicSquare)

```
WI-002: is_valid(grid)->bool, 합≠34→False · ECB: <<control>>
실패조건: 합≠34→False · 테스트: pytest AAA · PASS
```

1일차 강의 자료 · 8시간

워크 북 실습

D1_6.실습- 사용자 여정 작성
D1_7: 최종 PRD 작성 (Gherkin 포함)

1일차 핵심 정리

01

Prompt Engineering

P-C-T-F(기본) + 7단계(복잡한 설계) + Few-Shot/CoT/ReAct(고급)
Cursor AI 3모드: Agent(실행) · Plan(계획) · Ask(이해) · .cursorrules로 일관성 확보

02

C2C Traceability + 의사결정 파이프라인

문제인식→온톨로지(ECB)→프롬프팅 체인 = 의미 보존의 실천 공식
4계층: Concept→Responsibility→Test→Code — 설계 의도가 코드까지 살아있어야 한다

03

Magic Square × ECB × 문제인식

Entity(MagicSquare) · Control(SquareValidator) · Boundary(GridUI)
L3 실패 조건부터 시작: "언제 거부하는가"가 진짜 요구사항

04

Dual-Track TDD + Scenario + PRD

왜 늦게 실패? → PRD = 테스트의 원천 → Gherkin L0~L3 시나리오 → UX/Logic 분리
To-Do 구조: 체크박스·번호·추적·체크포인트·속성테스트·검증포인트

내일 2일차: TDD RED→GREEN→REFACTOR 완전 구현 · Cursor Agent 실전 활용 · Prompting Best Practices

THANK YOU

앞으로의 엔지니어는
단순한 '코더'가 아니라
AI와 협력하여 문제를 해결하는
지식 설계자입니다.